

1. feladat – Első egyszerű adatkötés**BindingContext = this****Fókusz:**

BindingContext, {Binding Property}, OneWay és TwoWay, INotifyPropertyChanged – MVVM használata nélkül.

Feladat:

Készítsen egy oldalt, amely tartalmaz:

- egy Entry vezérlőt,
- egy Label vezérlőt,
- egy Button vezérlőt.

Követelmények:

- Az oldal code-behind fájljában implementálja az INotifyPropertyChanged interfészt.
- Hozzon létre egy nyilvános: `string Name { get; set; }` tulajdonságot, amelynek setterében megtörténik a PropertyChanged esemény kiváltása.
- Az OnAppearing metódusban vagy a konstruktorban állítsa be: `BindingContext = this;`
majd inicializálja a Name értékét:
`Name = "Alice";`

XAML:

- Az `Entry.Text` tulajdonság `TwoWay` módban legyen a `Name` tulajdonsághoz kötve.
- A `Label.Text` tulajdonság `OneWay` módban legyen a `Name` tulajdonsághoz kötve.
- A `Button Clicked` eseménykezelője állítsa be: `Name = "Bob";`

Elvárt működés:

- A Label kezdetben az Alice szöveget jelenítse meg.
 - Az Entry mezőbe történő gépelés módosítsa a Label tartalmát.
 - A gomb megnyomásakor a Label tartalma változzon Bob értékre.
-

2. feladat – Binding módok és formázás

Binding módok, StringFormat és helyettesítő értékek

Fókusz:

Mode=OneWay, TwoWay, OneTime, Default, StringFormat, FallbackValue, TargetNullValue, statikus erőforrások – kizárólag az oldal code-behind fájljának használatával.

Feladat:

A code-behind fájlban hozzon létre az alábbi nyilvános tulajdonságokat:

- string? Username
- DateTime CreatedAt
- double Volume

XAML példák:

OneWay binding

```
Label.Text="{Binding Username, Mode=OneWay, FallbackValue=User: (unknown), TargetNullValue=User: (not set)}"
```

TwoWay binding

```
Entry.Text="{Binding Username, Mode=TwoWay}"
```

OneTime binding

```
Label.Text="{Binding CreatedAt, Mode=OneTime, StringFormat='Created: {0:yyyy-MM-dd HH:mm}'}"
```

Default binding – TwoWay

```
Slider.Value="{Binding Volume}"
```

- Hozzon létre egy pénznemjelet tartalmazó StaticResource erőforrást az App.xaml fájlban.
- Egy minta Label vezérlőben hivatkozzon rá a {StaticResource} használatával.
- Helyezzen el egy gombot, amelynek code-behind eseménykezelője először:

```
Username = null
```

értéket állít be, majd később:

```
Username = "Carol"
```

értéket, így megfigyelhetők a helyettesítő értékek.

Elvárt működés:

- Az egyes binding módok viselkedése legyen megfigyelhető.
- A dátum inicializálás után ne frissüljön.
- A Slider módosítása frissítse a Volume értékét.

3. feladat – Fordítási időben ellenőrzött bindingok és beágyazott tulajdonságok**x:DataType használata az oldal osztályával****Fókusz:**

Fordítási időben ellenőrzött bindingok (compiled bindings) használata x:DataType segítségével az oldal code-behind típusára; beágyazott tulajdonságok kötése; vezérlők közötti binding.

Feladat:**Code-behind**

Hozza létre az alábbi osztályt:

```
public class Person
{
    public string FirstName { get; set; } = "Ada";
    public string LastName { get; set; } = "Lovelace";
}
```

Hozzon létre egy nyilvános tulajdonságot:

```
public Person Current { get; set; } = new();
```

Ha a Current objektum teljes cseréjét is tervezi, implementálja az INotifyPropertyChanged működését az oldalon.

XAML

- A gyökérkonténeren állítsa be:

```
x:DataType="local:YourPageClassName"
```

Ezáltal a bindingok fordítási időben ellenőrizhetők az oldal típusával szemben.

- Helyezzen el Label vezérlőket, amelyek az alábbi tulajdonságokhoz kötődnek:
Current.FirstName
Current.LastName

- Valósítson meg vezérlők közötti bindingot:
 - hozzon létre egy Slider vezérlőt s1 néven,
 - egy Label jelenítse meg a Slider értékét az alábbi binding segítségével:
`{Binding Value, Source={x:Reference s1}, StringFormat='Value: {0:0}'}`
- Helyezzen el gombokat, amelyek code-behind kódból módosítják a `Current.FirstName` értékét.
- Ellenőrizze azt is, hogy a fordítási időben ellenőrzött binding hibás tulajdonságnév esetén már buildeléskor jelezze a problémát.

Elvárt működés:

- A beágyazott tulajdonságok bindingjai megfelelően működjenek.
- A `Current.FirstName` módosításakor frissüljön a felhasználói felület.
- Hibás bindingnév esetén a build folyamat figyelmeztetést adjon.

4. feladat – Converterek**Egyszeres és többszörös konverzió – Commandok nélkül, kizárólag eseményekkel****Fókusz:**

`IValueConverter`, `Toolkit MultiBinding`, valamint elemekből és erőforrásokból származó binding források.

Feladat:

A code-behind fájlban hozzon létre az alábbi nyilvános tulajdonságokat:

`decimal Price = 100m;`

`double Discount = 0.2;`

`bool InStock = true;`

Converterek

Hozza létre az alábbi convertereket:

PriceAfterDiscountConverter

Egyszeres értékű converter:

- bemeneti értéke a `Price`,
- a kedvezményt `double` típusú `ConverterParameter` értékként olvassa,
- a visszatérési érték:

Price * (1 - discount)

BoolToColorConverter

A konverzió:

- true → Green
- false → Red

XAML

A kedvezményes ár megjelenítésére szolgáló Label:

```
Text="{Binding Price, Converter={StaticResource PriceAfterDiscountConverter}, ConverterParameter={Binding Discount}, StringFormat='Final: {0:C}}'"
```

Egy állapotot megjelenítő Label:

```
TextColor="{Binding InStock, Converter={StaticResource BoolToColorConverter}}"
```

- Helyezzen el egyszerű gombokat Clicked eseménykezelőkkel.
- A gombok:
 - váltsák az InStock értékét, vagy
 - módosítsák a Discount / Price értékeket a code-behind fájlban.

Elvárt működés:

- Az árak és a színek frissüljenek a code-behind fájlban történő módosítások hatására.
- A hallgatók ismerjék meg az egyszeres és többszörös értékek konvertálását.

5. feladat – Gyűjtemények és kiválasztás

ObservableCollection használata code-behind eseményekkel

Fókusz:

ObservableCollection<T>, CollectionView.ItemsSource, SelectedItem TwoWay binding, valamint fordítási időben ellenőrzött DataTemplate.

Feladat:

Code-behind

Hozza létre az alábbi osztályt:

```
public class TodoItem  
{
```

```
public string Title { get; set; } = "";  
public bool Done { get; set; }  
}
```

Hozzon létre egy gyűjteményt:

```
ObservableCollection<TodoItem> Items = new();
```

Hozzon létre egy kiválasztott elemet tároló tulajdonságot:

```
TodoItem? SelectedItem { get; set; }
```

Szükség esetén váltsa ki a PropertyChanged eseményt.

- A konstruktorban inicializálja a gyűjteményt néhány elemmel.

XAML

- Helyezzen el egy szövegbeviteli mezőt: `Entry x:Name="newTitle"`
- Helyezzen el egy:

`Add`

feliratú gombot:

`Clicked="OnAdd"`

- Hozzon létre egy `CollectionView` vezérlőt:

`ItemsSource="{Binding Items}"`

`SelectedItem="{Binding SelectedItem, Mode=TwoWay}"`

DataTemplate

Használjon `DataTemplate`-et, opcionálisan `x:DataType="local:TodoItem"` beállítással.

A sablon jelenítse meg:

- egy `CheckBox` vezérlőben a `Done` tulajdonságot `IsChecked="{Binding Done}"` `TwoWay` bindinggal,
- valamint a `Title` értékét.

Helyezzen el a sablonban egy `Delete` feliratú gombot: `Clicked="OnDeleteClicked"`

Az eseménykezelőben az aktuális elem lekéréséhez használja:

```
((Button)sender).BindingContext
```

és kezelje azt aktuális `TodoItem` példányként.

- A lista alatt helyezzen el egy `Label` vezérlőt, amely az aktuálisan kiválasztott elem címét jeleníti meg:

```
{Binding SelectedItem.Title}
```

Code-behind eseménykezelők

OnAdd

Ha a newTitle.Text nem üres, adjon hozzá új elemet:

```
Items.Add(new TodoItem { Title = newTitle.Text });
```

OnDeleteClicked

Távolítsa el a BindingContext segítségével lekért aktuális elemet a gyűjteményből.

Elvárt működés:

- Az elemek hozzáadása és törlése működjön.
- A Done érték módosítása frissítse az alapul szolgáló objektumot.
- A kiválasztott elem címét megjelenítő Label automatikusan frissüljön.

6. feladat – Teljes körű „Katalógus + Kosár” alkalmazás

Fókusz:

Komplex binding forgatókönyv megvalósítása: beágyazott tulajdonságok, az oldalon tárolt gyűjtemények, számított végösszegek, elemek közötti és relatív bindingok – minden interakció code-behind eseménykezelők segítségével.

Feladat:

Code-behind modellek

Hozza létre a feladathoz szükséges modelleket.

Az oldal code-behind fájljának nyilvános tagjai

Szükség esetén INotifyPropertyChanged támogatással:

```
ObservableCollection<Product> Products = new();
```

```
ObservableCollection<CartLine> Cart = new();
```

```
decimal CartTotal { get; private set; }
```

A CartTotal értékét minden kosármódosítás után számítsa újra.

XAML elrendezés

1. Katalógus – CollectionView

A CollectionView:

ItemsSource = Products

értéket használjon.

A sablon jelenítse meg:

- a termék nevét,
- a beágyazott Category.Name tulajdonságot,
- az árat az alábbi formázással:

StringFormat='{0:C}'

Helyezzen el egy *Add* feliratú gombot: Clicked="OnAddToCart"

Az eseménykezelőben használja *((Button)sender).BindingContext* az aktuális Product objektum lekéréséhez, majd frissítse a Cart gyűjteményt a code-behind fájlban.

2. Kosár – CollectionView

A CollectionView *ItemsSource* = *Cart* értéket használjon.

A sablon jelenítse meg:

- Product.Name
- Product.Price
- Quantity
- a sor végösszegét.

A sor végösszegének megjelenítéséhez használjon:

- LineTotalConverter convertert a *{Binding .}* binding segítségével, vagy
- amennyiben külön tulajdonságot hoz létre hozzá, számítsa ki és formázza például *StringFormat='{0:C}'* használatával.

Helyezzen el:

- +
- –

gombokat az alábbi eseménykezelőkkel:

- Clicked="OnIncQty"
- Clicked="OnDecQty"

Az eseménykezelők a BindingContext segítségével kérjék le az aktuális CartLine objektumot, módosítsák annak Quantity értékét, majd számítsák újra a végösszeget.

Hasonló módon helyezzen el egy *Remove* gombot is.

3. Összesítő terület

Jelenítse meg a CartTotal értékét: *{Binding CartTotal, StringFormat='Total: {0:C}}'*

Optionálisan:

Használjon Toolkit MultiBinding megoldást a teljes összeg közvetlen kiszámításához a:

- Quantity,
- Product.Price

értékpárok alapján.

Ez alternatív megjelenítésként használható az eredmény összehasonlításához.

4. Kiegészítések

- Használjon x:Reference bindingot a kosárban található elemek számának megjelenítésére a kosárnézet *ItemsSource.Count* értékéből, vagy
 - tartson fenn az oldalon egy nyilvános *int CartCount* tulajdonságot.
 - Ha relatív bindingot is szeretne bemutatni, készítsen fejléceket, amely: *RelativeSource AncestorType=ContentPage* binding segítségével az oldal valamely szín- vagy címtulajdonságához kapcsolódik.
-

Kizárólag code-behind használata az eseménykezeléshez

- Minden gomb Clicked eseménykezelőt használjon.
- Minden kosármódosítás után:
 - számítsa újra a CartTotal értékét,
 - váltsa ki a PropertyChanged eseményt.

Elvárt működés:

- A katalógusból történő hozzáadás új elemet helyezzen a kosárba, vagy frissítse a már meglévő elemet.
- A + és – gombok módosítsák a mennyiséget.
- A végösszegek és elemszámok azonnal frissüljenek.

- A beágyazott bindingok (például: `Product.Category.Name`) megfelelően jelenjenek meg.
- Az opcionális `MultiBinding` segítségével számított végösszeg egyezzen meg a code-behind fájlban kiszámított végösszeggel.